

ES5 Fundamentals

Strict Mode · Hoisting · Scoping · Functions · Prototypes

Exercise 1 — Hoisting & Scoping Challenge

Difficulty: **Medium**

Analyze and fix the following code:

```
console.log(name);
var name = "Jone";

function test() {
  var x = 10;
  if (true) {
    var y = 20;
  }
  console.log(y);
}

test();
// console.log(x);
```

- Predict the output before running the code.
- Explain how hoisting works with var.
- Identify the difference between function scope and block scope.
- Rewrite the example using let where appropriate.

Exercise 2 — Constructor Functions & Prototypal Inheritance

Difficulty: **Medium**

Build a simple employee management system using ES5.

Requirements:

- Create a Person constructor with name and age.
- Add a greet() method to Person.prototype.
- Create an Employee constructor with employeeId and position.
- Make Employee inherit from Person using Object.create().
- Override the greet() method in Employee.prototype.
- Create at least three employees and demonstrate inheritance.

Arrays & JSON

concat · sort · splice · slice · reverse · includes · forEach · JSON

Exercise 3 — Array Methods Playground

Difficulty: **Easy**

Create an array containing at least 50 student names.

Requirements:

- Use `concat()` to combine two student arrays.
- Use `sort()` to sort the names alphabetically.
- Use `reverse()` to reverse the order.
- Use `includes()` to check whether a particular student exists.
- Use `forEach()` to print every student with their index.

Exercise 4 — Student Records Manager

Difficulty: **Medium**

Create an array containing **50 student objects**, where each object includes `id`, `name` and `grade`.

Requirements:

- Use `splice()` to add, remove and replace a student.
- Use `slice()` to create a copy of a portion of the array.
- Sort students by `grade`.
- Print the final list using `forEach()`.

Exercise 5 — JSON Converter

Difficulty: **Easy**

Create a JavaScript object representing a product.

Requirements:

- Include `id`, `name`, `price`, `category` and `available`.
- Convert the object to a JSON string using `JSON.stringify()`.
- Convert the JSON string back into a JavaScript object using `JSON.parse()`.
- Display the original object and converted object.
- Handle invalid JSON using `try...catch`

Exercise 6 — Product Inventory Analyzer

Difficulty: **Advanced**

Create an inventory containing at least ten products.

Requirements:

- Each product must have an `ID`, `name`, `price`, `category`, and `quantity`.
- Use `sort()` to order products by `price`.
- Use `includes()` to check a list of available product categories.
- Use `splice()` to remove a discontinued product.
- Use `slice()` to display the first five products.
- Use `concat()` to merge two inventories.

Modern JavaScript

let · const · Arrow Functions · Destructuring · Defaults · Spread · Rest · Map · Set

Exercise 7 — Arrow Function Transformation

Difficulty: **Easy**

Convert a set of traditional functions into arrow functions.

Requirements:

- Create a function that calculates the square of a number.
- Create a function that checks whether a number is even.
- Create a function that calculates the total price of products.
- Use arrow functions with `map()`, `filter()` and `reduce()`.

Exercise 8 — Destructuring & Default Parameters

Difficulty: **Medium**

Create a user profile management program.

Requirements:

- Create an object containing a user's name, email, age and address.
- Extract values using object destructuring.
- Rename at least one extracted property.
- Use array destructuring to extract values from a list of skills.
- Create a `createUser()` function with default parameter values.
- Demonstrate what happens when an optional parameter is omitted.

Exercise 9 — Spread, Rest, Map & Set Challenge

Difficulty: **Advanced**

Create a course enrollment management system.

Requirements:

- Use the spread operator to combine two arrays of enrolled students.
- Use the rest parameter to create a function that accepts any number of grades and calculates their average.
- Use Set to remove duplicate student IDs.
- Use Map to associate student IDs with their grades.
- Add, update, retrieve and delete entries in the Map.
- Convert the final student data into a regular array for display.

Templates, Classes & Modules

Template Literals · Classes · Inheritance · import / export

Exercise 10 — Dynamic Student Report

Difficulty: **Easy**

Create a student report generator using Template Literals.

Requirements:

- Create an array of at least five student objects.
- Generate a report for each student using Template Literals.
- Include the student's name, ID, grade and pass/fail status.
- Create a multiline report.
- Display the reports in the browser using the DOM.

Exercise 11 — Classes & Inheritance

Difficulty: **Medium**

Build a simple university management system using ES6 classes.

Requirements:

- Create a Person class with name, email and a getInfo() method.
- Create a Student class that extends Person.
- Create an Instructor class that extends Person.
- Use super() in the child constructors.
- Override at least one inherited method.
- Create instances of each class and demonstrate inheritance.

Exercise 12 — JavaScript Modules

Difficulty: **Medium**

Organize a student management application using ES modules.

Requirements:

- Create a students.js module containing student data and functions.
- Create a grades.js module containing grade calculation functions.
- Export named functions and at least one default export.
- Import the functions into app.js.
- Display student information in the browser.
- Run the application using a local development server and type="module".

Expected result: A working application split into reusable JavaScript modules.

Web Storage & Cookies

Web Storage · localStorage · sessionStorage · Cookies

Exercise 13 — Web Storage Methods

Difficulty: **Easy**

Build a browser-based storage playground.

Requirements:

- Use `setItem()` to save data.
- Use `getItem()` to retrieve data.
- Use `removeItem()` to delete a specific item.
- Use `clear()` to remove all items for the current origin.
- Use `key(index)` to retrieve a stored key.
- Use `length` to display the number of stored items.
- Display the current storage contents in the browser.

Exercise 14 — Local Storage To-Do List

Difficulty: **Medium**

Create a To-Do List application that saves tasks in `localStorage`.

Requirements:

- Add a new task.
- Display all saved tasks after refreshing the page.
- Mark tasks as completed.
- Delete individual tasks.
- Use `JSON.stringify()` and `JSON.parse()` to store an array of tasks.
- Display the number of tasks using the stored data.
- Add a button to clear all tasks.

Exercise 18 — Session Storage Multi-Step Form

Difficulty: **Medium**

Create a three-step registration form using `sessionStorage`.

Requirements:

- Step 1: Personal information.
- Step 2: Educational information.
- Step 3: Review and confirm.
- Save the user's progress in `sessionStorage`.
- Restore the current step when the page is refreshed within the same tab.
- Allow the user to navigate backward and forward.

Exercise 19 — Cookies & Preferences Manager

Difficulty: **Advanced**

Create a Cookie and user preferences manager.

Requirements:

- Create a function to set a cookie with an expiration date.

- Create a function to read a cookie by name.
- Create a function to delete a cookie.
- Add a theme preference (light/dark).
- Add a language preference (English/Arabic).

Display the saved preferences when the page loads.